

Mini-GPU

Daniel Chvat 

*Dept. of Electrical and Computer Engineering
University of California, Los Angeles
Los Angeles, CA, USA
dchvat@ucla.edu*

Luke Yamaguchi 

*Dept. of Electrical and Computer Engineering
University of California, Los Angeles
Los Angeles, CA, USA
yamaluke@g.ucla.edu*

Abstract—With the rapid growth of AI, massively parallel architectures have become dominant due to the inherently vectorized nature of core machine learning operations such as matrix multiplication, convolution, and large-scale tensor computation. As a result, there has been increasing interest in AI accelerators, including GPUs, TPUs, and NPUs. However, accelerator hardware design is largely dominated by companies such as NVIDIA and AMD, making detailed information about GPU architecture, hardware design, execution models, and security mechanisms difficult to access. This project addresses that gap through the development of Mini-GPU, an educational FPGA-based GPU accelerator designed to expose the core principles of modern parallel processors through an open and inspectable hardware/software stack.

Mini-GPU implements a simplified SIMT-style architecture with custom accelerator instructions, host-device communication, PyTorch runtime support, and a compiler pipeline for lowering CUDA-like programs into a custom ISA. The design is intentionally targeted toward limited-resource FPGA platforms, specifically the Basys 3 FPGA, which is widely used in university digital design and computer engineering courses, making the project accessible for educational use. In addition to exploring hardware and software design, Mini-GPU incorporates a hardware-enforced kernel verification mechanism using SHA-256, which ensures that only authenticated kernels may execute on the accelerator by comparing a digest of the uploaded instruction stream against a trusted golden hash stored on the FPGA.

Index Terms—FPGA, GPU architecture, SIMT, machine learning acceleration, hardware security.

I. INTRODUCTION

The rapid growth of artificial intelligence has created increasing demand for hardware capable of accelerating large-scale numerical workloads. Modern machine learning relies heavily on operations such as matrix multiplication, convolution, vector addition, and tensor operations, all of which are naturally suited to massively parallel execution. As a result, parallel architectures such as GPUs, TPUs, and NPUs have become central to both AI training and inference. Among these, GPUs remain especially important because of their widespread use in research, industry, and high-performance computing.

Despite their prevalence, modern GPU architectures are largely proprietary. Companies such as NVIDIA and AMD dominate the accelerator hardware ecosystem, and while high-level programming frameworks such as CUDA, ROCm, Py-

Torch, and TensorFlow make GPUs easier to use, they also hide many of the low-level architectural details that determine how computation is actually executed. Concepts such as SIMT execution, warp scheduling, memory hierarchy, instruction decoding, host-device communication, and secure accelerator interaction are often difficult to study directly without access to internal hardware documentation or expensive development test benches. This creates a gap between using AI accelerators and understanding how they work.

This project aims to address that gap through the development of *Mini-GPU*, an educational FPGA-based GPU accelerator designed to expose the core principles of parallel processor design through an open source hardware/software stack [1]. Rather than attempting to match the performance or complexity of commercial GPUs, Mini-GPU focuses on implementing a simplified but representative accelerator architecture that can be studied, modified, and extended. The system includes custom accelerator instructions, host-device communication, a compiler pipeline, and runtime support intended to connect the accelerator to PyTorch.

A major design goal of Mini-GPU is accessibility. The accelerator is targeted toward limited-resource FPGA platforms, specifically the Digilent Basys 3 FPGA, which is commonly used in university digital design and computer engineering courses. By designing around a constrained and widely available FPGA board, Mini-GPU makes GPU architecture concepts more approachable for students and researchers who may not have access to larger FPGA development boards or commercial accelerator design tools.

In addition to the proprietary nature of GPU hardware design, the security mechanisms used in modern AI accelerators are also often opaque. As machine learning models are increasingly deployed on specialized hardware, questions of integrity and trust become just as important as performance. However, it is difficult to study how commercial accelerators verify computation or detect tampering because many of these mechanisms are either undocumented or hidden behind proprietary hardware and software interfaces.

Mini-GPU addresses this issue by incorporating a hardware-enforced kernel verification mechanism directly into the accelerator design. Before any kernel is allowed to execute, the hardware computes a SHA-256 digest over the uploaded instruction stream and compares it against a trusted golden hash stored on the FPGA. If the digests do not match, the

launch is blocked and the system returns a fault to the host. By integrating this verification gate into the hardware/software interface, Mini-GPU provides a platform for studying not only accelerator architecture and compiler design, but also the challenge of ensuring that only authenticated code executes on specialized hardware.

II. RELATED WORK

Mini-GPU builds on prior work in open-source GPU design. One closely related project is Adam Majmudar’s *tiny-gpu*, a minimal Verilog GPU designed to demonstrate the fundamentals of GPU architecture from the ground up [2]. The project provides an implementation of instruction execution, memory access, and parallel computation. Mini-GPU extends the scope beyond a minimal compute architecture by incorporating a custom compiler flow, host-device communication, runtime support, PyTorch integration, mixed-precision arithmetic, and hardware security mechanisms.

Another relevant open-source GPU project is Vortex, a RISC-V based general-purpose GPU architecture designed for GPGPU research [3], [4]. Vortex provides a more complete and scalable accelerator platform with support for OpenCL. In contrast, Mini-GPU is intentionally designed for limited-resource FPGA platforms such as the Basys 3.

The security components of Mini-GPU are built around open-source cryptographic hardware IP. The project uses a modified version of the `secworks/sha256` Verilog SHA-256 core for kernel verification, which computes a digest over the uploaded instruction stream and compares it against a trusted golden hash stored on the FPGA to gate kernel launch [5].

Because detailed documentation for commercial GPU microarchitectures is limited, much of Mini-GPU was developed through a combination of public documentation, reverse engineering, CUDA programming behavior, and a general understanding of SIMT execution. The software-facing runtime and PyTorch integration were informed by PyTorch documentation on custom operators and C++/CUDA extensions [6]–[8].

III. METHODOLOGY

A. System Overview

The system is organized into three major layers: A FPGA hardware layer responsible for instruction execution, memory access, arithmetic operations, and security checks, a host-device communication layer responsible for transferring commands and data, and a software stack responsible for memory management, tensor allocation, compiling kernels, registering kernels, and launching kernels.

At the software level, CUDA-like kernel descriptions or PyTorch runtime calls are lowered into the Mini-GPU ISA. The host runtime then packages instructions, input data, launch commands, and readback requests into protocol packets. These packets are transmitted to the FPGA over UART. On the FPGA, a packet decoder and controller route incoming data to instruction memory, data memory, control registers, or security modules depending on the command type.

Once a kernel launch command is received, the execution controller begins fetching instructions from instruction memory and executing them using the accelerator datapath. After execution completes the accelerator notifies the runtime through status bits. Output data is stored in accelerator memory which the host can access using a pointer allocated by the runtime being tracked by PyTorch.

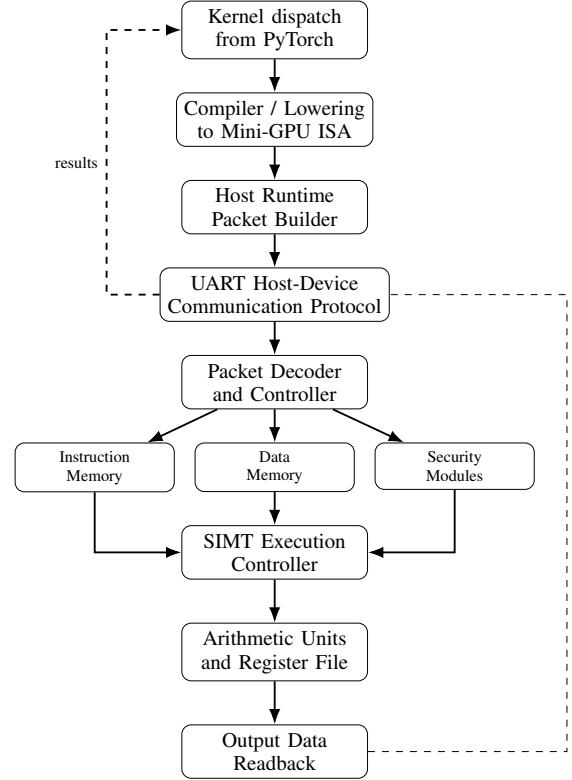


Fig. 1. High-level Mini-GPU system organization. The host software stack compiles kernels, builds protocol packets, and communicates with the FPGA over UART. The FPGA decodes packets, applies security checks, stores instructions and data, performs SIMT execution, and returns output data to the host.

B. FPGA Target and Resource Constraints

Mini-GPU targets the Digilent Basys 3, which is built around a Xilinx Artix-7 XC7A35T FPGA. The Artix-7 35T provides 20,800 LUTs, 41,600 flip-flops, 50 BRAM tiles of 36 Kb each, and 90 DSP48E1 slices. Its small form factor and low cost make it widely used in university digital design courses, though its constrained resources impose meaningful architectural tradeoffs on Mini-GPU.

LUTs are the primary binding constraint. With only 20,800 available, the combined logic of the SIMT execution controller, memory subsystem, communication protocol, and security modules must fit within a tight budget. This motivates area-conscious decisions throughout the design, including the use of approximation-based math routines rather than fully IEEE-compliant floating-point units, a compact custom ISA, and shared hardware resources wherever feasible.

BRAM capacity bounds the size of the three memory regions. The instruction memory, banked global data memory, and security hash storage must collectively fit within the 50 available 36 Kb tiles, placing a hard upper limit on kernel size, tensor capacity, and the number of storable kernel hashes. DSP48E1 slices are available for fixed-function multiply-accumulate operations, though their count of 90 is sufficient for the arithmetic units in the current design without requiring soft-logic fallbacks for the core datapath.

The host-device communication protocol is built on top of UART, which caps the transmission speed at a maximum baud rate of 115,200 bits per second. This bandwidth limitation, combined with the constrained instruction memory capacity, represents the most significant performance bottleneck in the system. Every kernel launch requires transferring the compiled instruction stream to the chip, and every operation requires transferring input tensors to device memory and reading output tensors back. Each of these transfers is serialized over the same low-bandwidth UART link. Furthermore, when a PyTorch operation internally dispatches multiple kernels and the combined instruction footprint exceeds the available instruction memory, the chip must stall and reload kernels mid-operation. This reload overhead compounds the base transfer cost and introduces significant latency for any workload that cannot fit all required kernels into program memory simultaneously.

C. Custom Instruction Set Architecture

The Mini-GPU ISA is a custom 32-bit fixed-width instruction set inspired by RISC-V, designed around the CUDA execution model. It uses fixed-width encoding, a load-store memory model, a regular three-register ALU format, and immediate variants for common arithmetic operations. Each instruction follows the format shown in Table I.

TABLE I
MINI-GPU 32-BIT INSTRUCTION FORMAT

[31:26]	[25:22]	[21:18]	[17:14]	[13:0]
OPCODE (6)	RD (4)	RS1 (4)	RS2 (4)	IMM14

The 6-bit opcode supports up to 64 instructions, the 4-bit register fields address 16 physical registers per lane, and the 14-bit signed immediate covers memory offsets and small constants. For typed ALU and memory instructions, the low three bits of `imm14` carry a data-format tag selecting among `I32`, `I16`, `I8`, `FP32`, `FP16`, and `FP8`, allowing a single opcode to operate across multiple precisions. Table II summarizes the instruction categories.

Dedicated thread metadata instructions such as `TID`, `TIDX`, `BID`, and `BDIM` expose the CUDA execution model directly in hardware, eliminating the need for the compiler to synthesize thread indexing from scalar operations. Memory operations are separated by address space, with `LDG/STG` targeting global memory, `LDS/STS` targeting per-block shared memory, and `LDC` loading kernel arguments from a constant table populated by the host runtime before launch. Divergent control flow is handled through `PUSHM`, `PRED`, `PREDN`, and `POPM`, which

TABLE II
MINI-GPU ISA INSTRUCTION CATEGORIES

Category	Instructions
Thread metadata	<code>TID</code> , <code>TIDX</code> , <code>BID</code> , <code>BDIM</code> , <code>GDIM</code> , <code>LID</code> , <code>WID</code>
Data movement	<code>MOV</code> , <code>MOVI</code> , <code>LDC</code>
Integer ALU	<code>ADD</code> , <code>ADDI</code> , <code>SUB</code> , <code>SUBI</code> <code>MUL</code> , <code>MULI</code> , <code>DIV</code> , <code>MOD</code>
Bitwise	<code>AND</code> , <code>ANDI</code> , <code>OR</code> , <code>ORI</code> , <code>XOR</code> , <code>XORI</code> , <code>NOT</code>
Shift	<code>SHL</code> , <code>SHLI</code> , <code>SHR</code> , <code>SHRI</code>
Comparisons	<code>SLT</code> , <code>SLE</code> , <code>SGT</code> , <code>SGE</code> , <code>SEQ</code> , <code>SNE</code>
Floating-point	<code>FADD</code> , <code>FSUB</code> , <code>FMUL</code> , <code>FDIV</code> , <code>FTOI</code> , <code>ITOF</code>
Memory	<code>LDG</code> , <code>STG</code> , <code>LDS</code> , <code>STS</code>
Predication	<code>PUSHM</code> , <code>PRED</code> , <code>PREDN</code> , <code>POPM</code>
Control flow	<code>BRA</code> , <code>BZ</code> , <code>BNZ</code> , <code>BAR</code> , <code>EXIT</code> , <code>NOP</code>

save, narrow, and restore the per-lane active mask around conditional bodies. Branches use signed PC-relative offsets encoded as `target_pc - (pc + 1)`, consistent with RISC-V convention.

D. SIMT Execution Model

Mini-GPU follows the single-instruction, multiple-thread (SIMT) execution model. The hardware is organized hierarchically as a core, streaming-multiprocessor-like execution unit, block, warp, and lane-level thread datapath. A warp is the basic instruction issue unit. Each lane executes the same decoded instruction for a different logical thread, but maintains its own register file, memory request state, execution state, and writeback path.

The SIMT hierarchy is shown in Fig. 2. The core maintains a program counter for each resident warp. On launch, each warp is initialized with the same base program counter, while its active-lane mask is derived from the launch mask and the block dimension. The scheduler scans the resident warps in round-robin order and selects the next warp that has not finished and is not waiting at a barrier. The selected warp's program counter is used to fetch one 32-bit instruction from program memory. After the instruction is fetched, it is issued to the SIMT datapath together with the selected block ID, warp ID, active mask, launch metadata, and constant-memory value. Thus, an instruction such as `ADD r3, r1, r2` is interpreted as four independent operations in a four-lane configuration:

$$r3_i \leftarrow r1_i + r2_i, \quad i \in \{0, 1, 2, 3\}.$$

The instruction is only presented to lane i when the corresponding bit in the warp's active mask is set.

Thread metadata is generated directly in hardware and is exposed through special ISA instructions such as `TID`, `TIDX`, `BID`, `BDIM`, `GDIM`, `LID`, and `WID`. For lane i in warp w of block b , the global thread identifier is computed as

$$tid = b \cdot block_dim + w \cdot W + i,$$

where W is the warp size. This allows kernels to compute element addresses in the same style as CUDA kernels, while the hardware only needs to supply per-lane metadata.

Control flow is handled using active masks. Mini-GPU provides `PUSHM` and `POPM` instructions to save and restore

the current active mask on a per-warp mask stack. Predicate instructions, such as `PRED` and `PREDN`, update the active mask using the per-lane condition results produced by the lanes. As a result, lanes whose predicate evaluates false stop receiving instructions until the saved mask is restored. Branch instructions update the selected warp’s program counter, while each warp keeps its own independent PC. The implementation also includes a small per-warp call stack for `CALL` and `RET`, and a block-level barrier mechanism for `BAR`. A warp that reaches a barrier is marked as waiting; once all unfinished warps in the block have reached the barrier or exited, the barrier is released and execution continues.

Memory operations are also issued in SIMT. Each active lane may generate an independent memory request from the same load or store instruction. The memory system receives one request per lane and routes requests to a four-bank data memory. Since the bank is selected from the low address bits, lanes accessing different banks can proceed in parallel. When several lanes target the same bank, the memory logic admits only the requests that can be served by the available bank ports and blocks the remaining lanes until their memory requests can be served. An instruction is considered complete only after all active lanes have completed their operation or produced the required writeback.

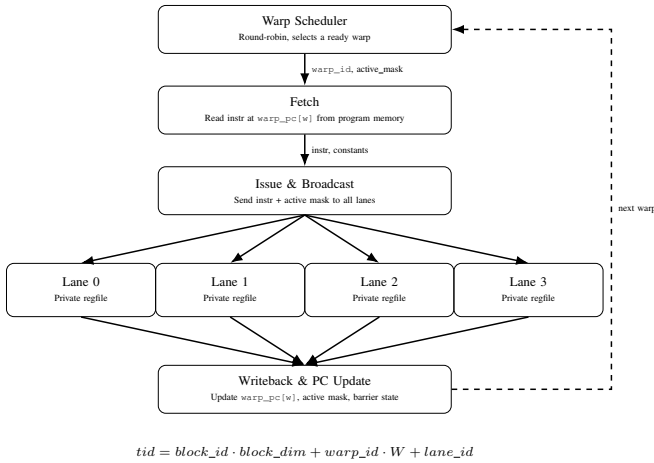


Fig. 2. Mini-GPU SIMT execution pipeline. The scheduler selects a ready warp, fetches the instruction at that warp’s PC, and broadcasts it with the active mask to all lanes. Each lane executes independently with its own register file. Results are collected and the per-warp PC, active mask, and barrier state are updated before the next warp is selected.

E. Memory Organization

Mini-GPU separates memory into three main regions: instruction memory, global data memory, and security memory. Instruction memory stores the Mini-GPU ISA program generated by the compiler and loaded by the host runtime before kernel launch. During execution, the SIMT controller fetches instructions sequentially using the program counter and decodes each instruction into arithmetic, memory, control, or security-related operations.

Global memory stores input tensors, output tensors, scalar arguments, and intermediate kernel data. To improve memory throughput, the global memory space is organized as a banked memory system. A word address is translated into a memory bank and row index using $bank = addr \bmod N_{banks}$ and $row = \lfloor addr / N_{banks} \rfloor$, where N_{banks} is the number of memory banks. This striped address mapping allows consecutive memory addresses to distribute across different banks, which improves throughput for vectorized memory accesses common in machine learning kernels. In the conflict-free case, the memory system can service up to one access per bank per cycle. If multiple logical threads access different banks, the requests can proceed in parallel. If two or more requests target the same bank in the same cycle, a bank conflict occurs. Mini-GPU handles these conflicts by serializing the conflicting requests and stalling or replaying the remaining accesses until the bank becomes available. Thus, peak throughput scales with the number of banks for coalesced access patterns, while worst-case throughput degrades when many lanes contend for the same bank.

Mini-GPU includes a dedicated security memory region for storing precomputed hashes of trusted kernel instruction blocks. Each security memory entry stores a 256-bit SHA-256 hash. This memory is indexed by a kernel identifier or instruction-block address and is used during verification to compare the expected kernel hash against the hash computed over the loaded instruction stream.

Fig. 3 shows the overall logical memory layout, and Fig. 4 illustrates banked global memory access and conflict behavior.

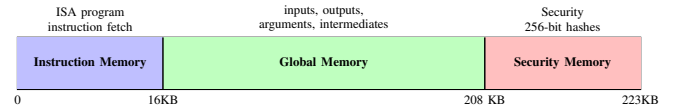


Fig. 3. Logical Mini-GPU memory map showing the three primary memory regions: instruction memory, global memory, and security memory.

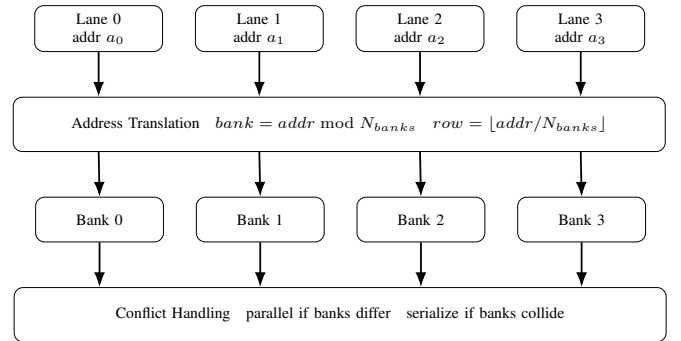


Fig. 4. Banked global memory access. SIMT lanes generate addresses, which are translated into bank and row indices. Requests to different banks proceed in parallel; requests to the same bank are serialized.

F. Arithmetic and Mixed-Precision Support

The supported data formats are shown in Table III. Integer operations support 32-bit, 16-bit, and 8-bit signed values. Floating-point operations support FP32, FP16, and FP8 formats.

TABLE III
MINI-GPU ARITHMETIC FORMATS.

Format	Use
I32	32-bit signed integer
I16	16-bit signed integer
I8	8-bit signed integer
FP32	32-bit floating point
FP16	16-bit floating point
FP8	8-bit floating point

G. Host-Device Communication Protocol

Mini-GPU communicates with the host through a lightweight UART-based command protocol. The host-side runtime is responsible for opening the serial device, packetizing commands, transferring kernel and tensor data, launching execution, and reading results back from the FPGA. The FPGA receives these packets through the UART control path, decodes the command field, and routes each request to instruction memory, global data memory, constant memory, the launch controller, or the secure kernel verification logic.

Each protocol packet begins with a fixed start-of-frame byte, followed by a command, address field, length field, optional payload, and checksum. The packet format is shown in Fig. 5. Each packet begins with an 8-bit start-of-frame byte, followed by an 8-bit command, a 16-bit address field, a 16-bit length field, an optional payload of up to 255 bytes, and an 8-bit checksum.

UART Packet Format

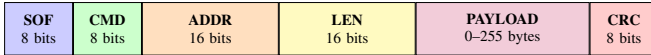


Fig. 5. Mini-GPU UART packet format. Each packet begins with an 8-bit start-of-frame byte, followed by an 8-bit command, 16-bit address, 16-bit length, variable-length payload, and 8-bit checksum.

The start-of-frame byte is $0xAA$. The address field is 16 bits and is transmitted most-significant byte first. The length field is also 16 bits and specifies the number of payload bytes for write commands or the requested number of bytes for read commands. The checksum is a one-byte XOR CRC computed over every byte from CMD through the final payload byte. Thus, for a packet buffer p , the CRC is

$$CRC = p_1 \oplus p_2 \oplus \dots \oplus p_{n-1},$$

where p_0 is the start-of-frame byte and p_n is the transmitted CRC byte. This checksum is simple enough for compact FPGA logic while still detecting common framing and byte-corruption errors on the UART link.

The command set is summarized in Table IV. The protocol separates memory movement, kernel loading, execution control, and security operations. Data-memory commands are used to move tensors and intermediate buffers between the host and Mini-GPU global memory. Program-memory commands are used to load compiled Mini-GPU instruction words before launch. Constant-memory commands transfer scalar arguments and kernel parameters. Security commands reset the verification state and trigger validation of the uploaded kernel before execution is allowed.

TABLE IV
MINI-GPU HOST-DEVICE COMMAND SET.

Command	Value	Description
WRITE_DATA	0x01	Write payload bytes to data memory
READ_DATA	0x02	Read bytes from data memory
WRITE_PROGRAM	0x03	Write kernel instructions
LAUNCH	0x04	Start kernel execution
READ_STATUS	0x05	Read device status
SECURITY_RESET	0x06	Reset security state
VALIDATE	0x07	Validate uploaded kernel hash
ACK	0x08	Positive acknowledgement
NAK	0x09	Negative acknowledgement
WRITE_CONSTANTS	0x0B	Write constant memory

Large transfers are split into multiple packets by the host runtime. The maximum payload size is 255 bytes, while write transfers are padded and aligned to 32-bit words before transmission. This alignment matches the internal memory organization of Mini-GPU and avoids partial-word ambiguity at the hardware interface. For each write packet, the host waits for an acknowledgement from the FPGA. A valid acknowledgement must contain the ACK command, the matching address, and a zero-length payload. If the acknowledgement is missing, malformed, or replaced by a NAK, the host retries the transfer up to a fixed retry limit before reporting an error.

Read transactions use a request-response sequence. The host first sends a READ_DATA packet with an address and requested length but no payload. The FPGA responds with a packet containing the requested data bytes. The host verifies the start-of-frame byte, length, command, address, and CRC. If the response is valid, the host sends an ACK; otherwise, it sends a NAK and retries the request.

H. Software Stack

Mini-GPU includes a complete software stack that connects high-level tensor operations to execution on the FPGA. The software handles kernel compilation, binary loading, buffer management, host-device communication, kernel security validation, execution control, and framework-level dispatch.

At the lowest software level, the communication library implements the host-to-FPGA transport protocol in C. Mini-GPU uses a UART link rather than a high-bandwidth interface such as PCIe or AXI DMA. The communication library is responsible for opening the serial device, constructing protocol packets, computing the checksum, transmitting commands, and checking device responses. It provides APIs to handle the basic operations used by the rest of the stack, including writing instruction memory, writing and reading global memory,

transferring constants, issuing validation commands, launching kernels, and reading device status.

Above the communication layer is the C++ host runtime. The runtime acts as the main device-management layer for Mini-GPU. It tracks device state, manages host-side representations of Mini-GPU buffers, loads compiled kernel binaries, transfers input and output tensors, and coordinates execution. Before a kernel launch, the runtime uploads the compiled instruction stream into program memory, writes scalar arguments and launch metadata into constant memory, and copies input tensors into global memory. After execution completes, the runtime reads output buffers back from Mini-GPU global memory and returns them to the application. This layer also integrates with the secure kernel-launch mechanism by maintaining the trusted-kernel flow and invoking validation before execution.

The compiler toolchain translates kernels into the custom Mini-GPU ISA. Conceptually, the compiler follows a multi-stage lowering flow:

Kernel Source $\rightarrow$ AST $\rightarrow$ IR $\rightarrow$ Mini-GPU ISA $\rightarrow$ Binary.

The output of this process is a program image that can be written into Mini-GPU instruction memory.

The top software layer integrates Mini-GPU with PyTorch. The PyTorch backend registers Mini-GPU as a custom PrivateUser1 device and dispatches supported tensor operations to the FPGA through the runtime layer. Operations such as addition, matrix multiplication, ReLU, and convolution can therefore be expressed using familiar tensor-level APIs while being executed by Mini-GPU kernels. When a supported operation is invoked, the PyTorch layer prepares tensor metadata and buffer pointers, calls into the C++ extension and runtime, and receives the output tensor after FPGA execution completes. Unsupported operations can remain on the host or fall back to other execution paths.

I. Kernel Execution

On the FPGA, the packet decoder receives the serialized byte stream and routes each command to the appropriate hardware component. Instruction packets are written into instruction memory, data and argument packets are written into data memory or control registers, and launch packets configure the execution controller. Once the launch command is received, the controller initializes the program counter, marks the accelerator as busy, and begins fetching Mini-GPU instructions.

When all logical threads complete, the accelerator writes output data to memory and updates status bits indicating completion. The runtime then polls or reads the status register, retrieves output data, and returns the result as a PyTorch-visible tensor.

Fig. 6 shows the complete execution path for a vector add example.

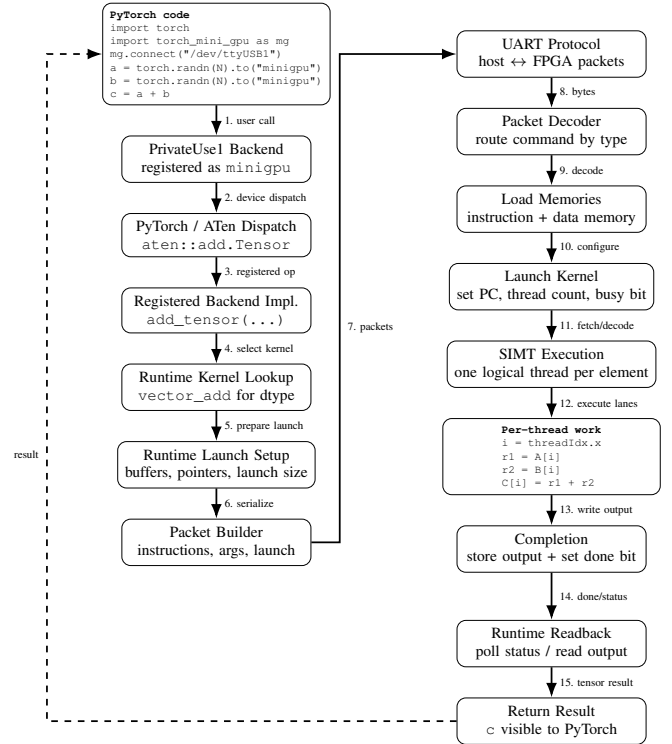


Fig. 6. Example Mini-GPU kernel execution flow for $c = a + b$. Tensors moved to the minigpu device are dispatched through PyTorch's PrivateUser1 backend. The `aten::add.Tensor` operation calls the registered Mini-GPU backend implementation, which resolves the runtime `vector_add` kernel, packetizes the launch, transfers commands to the FPGA over UART, executes the kernel, and returns the result to PyTorch.

J. Hardware Security Extensions

Mini-GPU incorporates a hardware-enforced kernel verification mechanism implemented as a dedicated security module called `kernel_vfy`. The security goal is that only authenticated kernels may execute on the accelerator, and that verified instruction memory cannot be modified while execution is active.

The trust boundary is defined as follows. Trusted hardware includes the security FSM, the SHA-256 core, and the golden hash BRAM. Everything outside this boundary is considered untrusted, including the host software, the UART communication channel, uploaded kernel bytes, and data memory. The accelerator core is only permitted to execute a kernel if the instruction stream was fully uploaded, hashed, matched against a trusted golden hash, and instruction memory was subsequently locked.

1) *Verification FSM*: The `kernel_vfy` module implements a seven-state FSM that gates both program writes and kernel launches, as shown in Fig. 7. In the **IDLE** state no verified kernel is active and the host may begin a kernel upload via `COM_CMD_WRITE_PROGRAM`. During **LOAD**, each accepted instruction word is simultaneously written to instruction memory and fed into the SHA-256 engine. A SHA-256 fault or CRC failure during this phase immediately drives the FSM to **ERROR**. On upload com-

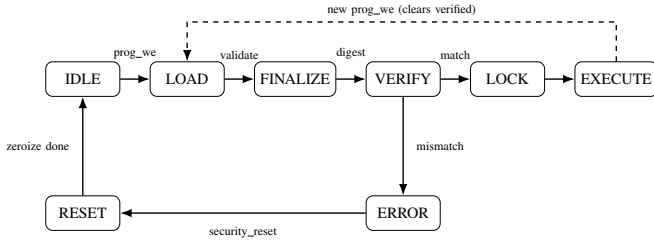


Fig. 7. The `kernel_vfy` security FSM. The dashed arc shows the re-entry path from EXECUTE back to LOAD when the core is idle and a new kernel upload begins. A hash mismatch drives the FSM to ERROR, from which recovery requires a security reset that zeroizes all instruction memory before returning to IDLE.

pletion the host sends `COM_CMD_VALIDATE`, moving the FSM to **FINALIZE** where the SHA-256 wrapper pads and finalizes the digest, then to **VERIFY** where the computed digest is compared against the golden hash selected by the kernel ID in the validate command. A match drives the FSM through **LOCK**, which disables further program writes and asserts `kernel_verified`, then into **EXECUTE**, where the launch gate opens and the core may fetch from locked instruction memory. A hash mismatch in VERIFY similarly drives the FSM to **ERROR**, blocking all launches and program writes until the host issues a `SECURITY_RESET`. If the core is idle in EXECUTE and a new `COM_CMD_WRITE_PROGRAM` is received, the FSM clears the verified state and returns to LOAD for a new kernel upload.

The design enforces two independent hardware gates. The program write gate blocks all instruction memory writes after LOCK, during ERROR, and whenever the core is busy, preventing the loaded instruction stream from being tampered with mid-execution. The launch gate permits a kernel launch only when `state == EXECUTE` and `kernel_verified == 1`, ensuring that execution cannot begin unless the full verification sequence has completed successfully.

2) *Golden Hash Storage:* Trusted golden hashes are stored in a dedicated BRAM region pre-loaded from a `.mem` file at bitstream time. The BRAM is indexed by a 7-bit kernel ID, supporting up to 128 entries. The kernel ID to hash mapping and the `.mem` file are generated automatically by a build-time script that reads a `kernels.yaml` configuration file and computes SHA-256 digests over each compiled kernel binary, eliminating manual hash management.

3) *Security Reset and Zeroization:* An early analysis of the security design identified a residual instruction memory vulnerability: a security reset that only cleared verification state without clearing instruction memory would allow an attacker to upload a corrupted kernel, deliberately fail validation to trigger a reset, and then launch from the stale corrupted instruction stream after recovery. To close this attack vector, the security reset was extended to zeroize the entire instruction memory by iterating through all 2^{12} program addresses and writing zero before returning to IDLE. This is implemented as a dedicated RESET state in the FSM. The zeroization

logic required 52 additional LUTs, a 2.4% increase over the pre-zeroization `kernel_vfy` module, and fit within the remaining LUT budget on the Basys 3.

IV. MATHEMATICAL FUNCTION APPROXIMATIONS

Many machine learning kernels rely on functions such as reciprocal, square root, exponential, logarithm, trigonometric functions, sigmoid, and hyperbolic tangent. Implementing each of these directly in hardware would require significant FPGA resources. To reduce area and simplify the hardware/software interface, Mini-GPU provides a lightweight device-side math library that approximates these functions using integer bit manipulation, polynomial approximations, range reduction, and iterative refinement.

The approximation library is implemented as a set of shared device functions that can be called by Mini-GPU kernels. Several helper routines reinterpret floating-point values as unsigned integers and use bit masks to manipulate sign, exponent, and mantissa fields directly. This allows functions such as absolute value, sign copying, special-case handling, and branchless selection to be implemented using simple integer operations. For example, `fabsf` clears the sign bit of the input, while `copysignf` combines the magnitude bits of one value with the sign bit of another. Similarly, `minigpu_select_u32` and `minigpu_select_f32` implement mask-based conditional selection, allowing special cases such as zero, infinity, and NaN to be handled without requiring complex control flow.

A. Basic Floating-Point Helpers

Mini-GPU implements several basic functions directly through integer reinterpretation and simple comparisons. Absolute value is computed by clearing the most significant sign bit. Minimum and maximum are computed by comparing floating-point values using an ordering key derived from their bit representation. This allows comparisons to work consistently across positive and negative values while avoiding a full floating-point comparison unit. Rounding functions are approximated using integer casts: `truncf` casts the input to an integer and back to a float, `roundf` adds or subtracts 0.5 depending on the sign before casting, and `floorf` and `ceilf` adjust the truncated integer result when the casted value lies above or below the original input.

B. Reciprocal and Reciprocal Square Root

The reciprocal function `rcpf` approximates $1/x$ using an initial estimate derived from the floating-point bit representation, followed by Newton-Raphson refinement. The initial estimate is computed by subtracting the magnitude bits of the input from a constant:

$$y_0 = \text{reinterpret}(C - \text{bits}(x)),$$

where C is a bit-level approximation constant. The estimate is then refined using the Newton-Raphson update for reciprocal:

$$y_{n+1} = y_n(2 - xy_n).$$

Mini-GPU applies this update multiple times to improve accuracy. Special cases are handled explicitly: reciprocal of zero returns signed infinity, reciprocal of infinity returns signed zero, and NaN inputs are propagated.

The reciprocal square root function `rsqrtf` follows a similar strategy. It uses the well-known inverse square root bit-level seed

$$y_0 = \text{reinterpret}(0x5f3759df - (\text{bits}(x) >> 1)),$$

followed by Newton-style refinement:

$$y_{n+1} = y_n (1.5 - 0.5xy_n^2).$$

The square root function is then computed using the identity

$$\sqrt{x} = x \cdot \frac{1}{\sqrt{x}}.$$

Additional masking logic handles negative inputs, zero, infinity, and NaN cases.

C. Logarithms

The logarithm implementation is based on decomposing a floating-point number into its exponent and mantissa. For `log2f`, the input is first separated into an integer exponent term k and a normalized mantissa $y \in [1, 2)$. The logarithm is then written as

$$\log_2(x) = k + \log_2(y).$$

To approximate the mantissa term, Mini-GPU uses the transformation

$$z = \frac{y - 1}{y + 1},$$

and approximates the natural logarithm using a truncated odd-power expansion:

$$\ln(y) \approx 2 \left(z + \frac{z^3}{3} + \frac{z^5}{5} + \frac{z^7}{7} \right).$$

The result is converted to base two using

$$\log_2(y) = \frac{\ln(y)}{\ln(2)}.$$

The natural logarithm and base-10 logarithm are then implemented through constant scaling:

$$\log(x) = \ln(2) \log_2(x),$$

$$\log_{10}(x) = \log_{10}(2) \log_2(x).$$

The implementation also handles zero, negative values, infinity, subnormal inputs, and NaN values through explicit bit-mask checks.

D. Exponentials and Power

The exponential approximation uses the identity

$$e^x = \left(1 + \frac{x}{N}\right)^N$$

for a large power-of-two value N . In Mini-GPU, the implementation uses $N = 2048$, since

$$\frac{1}{2048} = 0.00048828125.$$

The initial value is computed as

$$y = 1 + \frac{x}{2048},$$

and then repeatedly squared eleven times to approximate e^x :

$$e^x \approx \left(1 + \frac{x}{2048}\right)^{2048}.$$

Overflow, underflow, and NaN cases are handled using bit-level masks. The base-two exponential is approximated using

$$2^x = e^{x \ln 2},$$

and the power function is implemented through the identity

$$\text{pow}(a, b) = e^{b \log(a)}.$$

This design avoids implementing a separate hardware unit for each exponential variant.

E. Trigonometric Functions

The sine and cosine approximations use range reduction followed by low-order polynomial evaluation. First, the input is mapped to the nearest quadrant by computing

$$q = \text{round}\left(\frac{2x}{\pi}\right).$$

The reduced angle is then

$$r = x - q \frac{\pi}{2}.$$

This places the input into a smaller interval where low-order Taylor approximations are more accurate. Mini-GPU approximates sine using

$$\sin(r) \approx r - \frac{r^3}{6} + \frac{r^5}{120} - \frac{r^7}{5040},$$

and cosine using

$$\cos(r) \approx 1 - \frac{r^2}{2} + \frac{r^4}{24} - \frac{r^6}{720}.$$

The quadrant value determines whether the sine or cosine polynomial should be used and whether the sign should be flipped. This is implemented using mask-based selection and sign-bit manipulation. Tangent is computed as

$$\tan(x) = \frac{\sin(x)}{\cos(x)},$$

using the reciprocal approximation for division.

F. Activation Functions

Mini-GPU also provides approximations for activation functions commonly used in machine learning. The sigmoid function is computed using

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

where the exponential and reciprocal are provided by the approximation routines described above. To improve behavior for large inputs, the implementation saturates large positive values to 1 and large negative values to 0.

The hyperbolic tangent function is implemented in terms of sigmoid:

$$\tanh(x) = 2\sigma(2x) - 1.$$

For large magnitudes, the output is saturated to either 1 or -1 depending on the sign of the input. This is appropriate for many machine learning workloads because activations such as sigmoid and tanh naturally saturate for large positive or negative inputs.

V. RESULTS

A. FPGA Resource Utilization

The final Mini-GPU design was synthesized and implemented in Vivado v2018.2 targeting the Digilent Basys 3 Artix-7 XC7A35T using the `Flow_AreaOptimized_high` synthesis strategy and the `Performance_Explore` implementation strategy. The design closes timing at 75 MHz with a WNS of 0.020 ns. Table V summarizes the final resource utilization.

TABLE V
FINAL MINI-GPU RESOURCE UTILIZATION ON ARTIX-7 XC7A35T AT 75 MHz

Resource	Used	Available	Utilization
LUT	20,275	20,800	97.5%
FF	17,965	41,600	43.2%
BRAM (36 Kb)	40	50	80.0%
DSP48E1	19	90	21.1%

LUT utilization is the tightest constraint at 97.5%, leaving minimal headroom for additional logic. BRAM utilization is also significant at 80%, consumed by instruction memory, the four-bank interleaved global data memory, and the security hash storage region. DSP utilization remains modest at 21.1% because the design relies primarily on soft-logic arithmetic for the approximation-based math library rather than requiring a large number of dedicated multiply-accumulate blocks. Total on-chip power at 75 MHz is 0.244 W, which remains within the Basys 3 USB power budget.

The critical path in the final design is located in the memory subsystem, which is the same bottleneck that prevented the design from closing timing at the original 100 MHz target. Reducing memory access latency to reach 100 MHz operation is an identified direction for future work.

B. Kernel Correctness

Mini-GPU was validated against PyTorch CPU reference outputs for a set of core tensor operations. The operations tested include vector addition, matrix multiplication, element-wise ReLU activation, and 2D convolution. For each operation, output tensors produced by the Mini-GPU backend were compared against the corresponding PyTorch CPU results using an absolute tolerance threshold to account for differences introduced by the approximation-based math library and reduced-precision arithmetic.

All tested operations produced results within acceptable tolerance of the CPU reference, confirming functional correctness of the SIMT execution pipeline, memory subsystem, compiler lowering, and PyTorch dispatch integration across the supported operation set.

C. MNIST Training on Mini-GPU

To validate the full hardware/software stack end to end, a small convolutional neural network was trained on the MNIST handwritten digit dataset [9] through the Mini-GPU PyTorch backend. The model consists of a convolutional layer followed by two fully connected layers with ReLU activations, kept intentionally small to fit within the memory constraints of the Basys 3. Training used SGD with a learning rate of 0.03 and a batch size of 1, with the full forward pass, cross-entropy loss, and backward pass all dispatched to the FPGA.

A full epoch over the 60,000 training samples could not be completed within a practical time budget due to the UART bandwidth bottleneck. After running overnight the training loop was canceled before finishing one epoch. Despite this, the run confirmed end-to-end functional correctness: loss was decreasing and training accuracy had reached approximately 68% at the point of cancellation, confirming that the forward-backward-update cycle was executing correctly on hardware.

VI. FUTURE WORK

A. Memory Subsystem Optimization and Clock Frequency

The critical path in the current Mini-GPU implementation is located in the memory subsystem, which prevented the design from closing timing at the target 100 MHz clock frequency and required reducing the operating clock to 75 MHz. Future work should focus on optimizing the RTL of the memory modules to reduce the critical path delay and achieve timing closure at 100 MHz. Potential approaches include pipelining the memory access path, restructuring the banked memory address translation logic, and exploring alternative BRAM configurations that reduce combinational depth on the read data path.

B. Security Enhancements

1) *Validated Memory Regions*: The current design does not enforce validated memory regions in instruction memory. An attacker with repeated UART access could upload and validate a kernel at one address, then upload a second kernel that partially overwrites the first, and repeat this process to assemble a malicious instruction stream that is never individually

validated as a whole. Two approaches were analyzed during development. The first is to restrict all program writes to start at address 0×0000 and salt the hash with the write address, which provides tight security but eliminates kernel caching and imposes a significant performance penalty due to the UART bandwidth constraint. The second is a region tracking scheme that records which instruction memory regions are validated and invalidates any region that receives a subsequent write, allowing multiple kernels to coexist in instruction memory while preventing partial overwrite attacks. The region tracking approach is the preferred direction but requires significant new logic that exceeded the remaining LUT budget on the Basys 3. A target platform with more available LUTs, or an optimized memory subsystem that frees up resources, would make this approach viable.

2) *Secure Host-Device Communication*: The current host-device communication channel is unencrypted, meaning that kernel binaries, input tensors, and output tensors are transmitted in plaintext over UART. During development, AES encryption and HMAC-based message authentication were prototyped and evaluated as candidates for securing this channel. AES encryption would protect the confidentiality of kernel binaries and model weights in transit, while HMAC authentication would allow the FPGA to verify that packets have not been tampered with in transit and originate from a trusted host. These mechanisms were not included in the final design due to the LUT constraints of the Basys 3, but the prototyping work demonstrated their feasibility. Integrating both into a future revision of the communication protocol would complete the security boundary from end to end, complementing the existing on-device kernel verification with protection of the communication channel itself.

C. Communication Bandwidth

The UART-based host-device communication protocol is the primary performance bottleneck of the current system, capping transfer speed at 115,200 baud and making kernel load times and tensor transfers the dominant cost for most workloads. Replacing UART with a higher-bandwidth interface such as USB or PCIe would dramatically reduce transfer latency and make the system viable for more meaningful training and inference workloads. A higher-bandwidth interface would also reduce the impact of the kernel reload overhead that occurs when multiple kernels required by a single PyTorch operation cannot all fit in instruction memory simultaneously.

REFERENCES

- [1] D. Chvat and L. Yamaguchi, "Mini-GPU: An educational FPGA-based GPU accelerator," <https://github.com/DanielChvat/Mini-GPU>, 2026.
- [2] A. Majmudar, "tiny-gpu: A minimal gpu design in verilog to learn how gpus work from the ground up," <https://github.com/adam-maj/tiny-gpu>, 2024.
- [3] F. Elsabbagh, B. Tine, P. Roshan, E. Lyons, E. Kim, D. E. Shim, L. Zhu, S. K. Lim, and H. Kim, "Vortex: Opencl compatible risc-v gpgpu," <https://arxiv.org/abs/2002.12151>, 2020.
- [4] B. Tine, F. Elsabbagh, K. Yalamarthy, and H. Kim, "Vortex: Extending the risc-v isa for gpgpu and 3d-graphics research," <https://arxiv.org/abs/2110.10857>, 2021.
- [5] J. Strömbergson, "Sha-256: Hardware implementation of the sha-256 cryptographic hash function," <https://github.com/secworks/sha256>, secworks cryptographic hardware IP core.
- [6] PyTorch Contributors, "Custom c++ and cuda operators," https://docs.pytorch.org/tutorials/advanced/cpp_custom_ops.html.
- [7] PyTorch Contributors, "Pytorch custom operators," https://docs.pytorch.org/tutorials/advanced/custom_ops_landing_page.html.
- [8] PyTorch Contributors, "Facilitating new backend integration by privateuse1," <https://docs.pytorch.org/tutorials/advanced/privateuseone.html>.
- [9] Y. LeCun, C. Cortes, and C. J. Burges, "The MNIST database of handwritten digits," 1998. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>